Name: Mahathi A J
Roll: 241801148

First-In,First-Out CPU Scheduling Algorithm

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.
2. The next lines contain two integers for each process, where the first integer represents the Arrival Time (AT) and the second integer represents the Burst Time (BT).

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

3

0 2

1 2

2 3

Sample Output 1

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33

Average Waiting Time: 1.00

Example 2

Sample Input 2

3

0 5

5 3

8 2

Sample Output 2

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33

Average Waiting Time: 0.00

Name: Mahathi A J
Roll: 241801148

Answer:

```
#include <stdio.h>

int main() {
int n, i;
printf("Enter number of processes: ");
scanf("%d", &n);

int AT[n], BT[n], CT[n], TAT[n], WT[n];

// Input Arrival Time and Burst Time
for(i = 0; i < n; i++) {
printf("\nEnter Arrival Time and Burst Time for P%d: ", i + 1);
scanf("%d %d", &AT[i], &BT[i]);
}
// First process completion time
CT[0] = AT[0] + BT[0];

// Calculate Completion Time for remaining processes
for(i = 1; i < n; i++) {
if(CT[i-1] < AT[i]) {
// CPU is idle
CT[i] = AT[i] + BT[i];
} else {
CT[i] = CT[i-1] + BT[i];
}
}
// Calculate TAT and WT
float totalTAT = 0, totalWT = 0;

for(i = 0; i < n; i++) {
TAT[i] = CT[i] - AT[i];
WT[i] = TAT[i] - BT[i];

totalTAT += TAT[i];
totalWT += WT[i];
}
// Calculate Averages
float avgTAT = totalTAT / n;
float avgWT = totalWT / n;

printf("\n\nAverage Turnaround Time: %.2f\n", avgTAT);
printf("Average Waiting Time: %.2f\n", avgWT);

return 0;

}
```

Name: Mahathi A J
Roll: 241801148

Operating System

OUTPUT:

Custom Test — Success

Input :

```
3
0 2
1 2
2 3
```

Expected Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00
```

Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00
```

Mahathi A J
241801148

Operating System

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- $Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)$
- $Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)$

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The next lines contain two integers for each process, where the first integer denotes the Arrival Time (AT) of the process and the second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

4

0 6

0 8

0 7

0 3

Sample Output 1

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00

Average Waiting Time = 7.00

Example 2

Sample Input 2

3

0 3

5 2

5 1

Sample Output 2

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Average Turnaround Time = 2.33

Average Waiting Time = 0.33

Mahathi A J
241801148

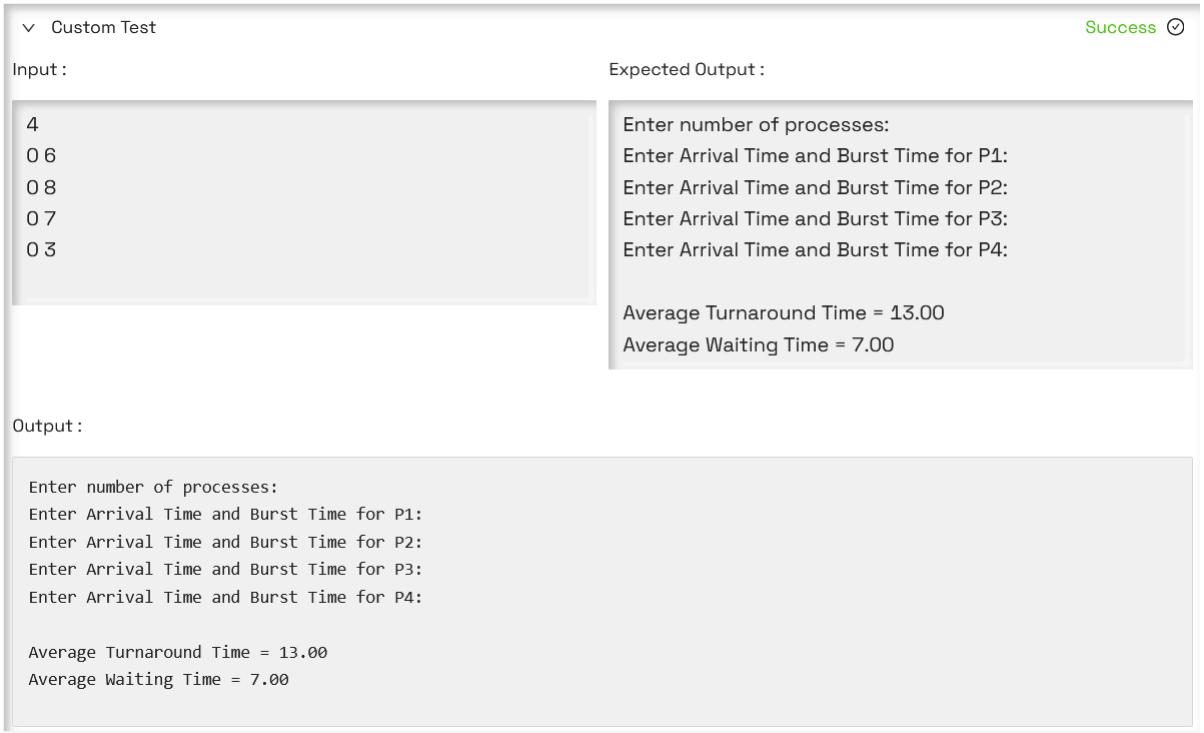
Operating System

Answer:

```
#include <stdio.h>

int main() {
int n, i, j, time = 0, completed = 0, min_index;
printf("Enter number of processes: ");
scanf("%d", &n);

int AT[n], BT[n], CT[n], TAT[n], WT[n], visited[n];

// Input
for(i = 0; i < n; i++) {
printf("\nEnter Arrival Time and Burst Time for P%d: ", i + 1);
scanf("%d %d", &AT[i], &BT[i]);
visited[i] = 0; // Not executed yet
}

// SJF Scheduling Logic
while(completed < n) {
min_index = -1;

// Find process with minimum BT among arrived processes
for(i = 0; i < n; i++) {
if(AT[i] <= time && visited[i] == 0) {
if(min_index == -1 || BT[i] < BT[min_index]) {
min_index = i;
}
}
}

// If no process has arrived, move time forward
if(min_index == -1) {
time++;
} else {
time += BT[min_index];
CT[min_index] = time;
visited[min_index] = 1;
completed++;
}
}

// Calculate TAT and WT
float totalTAT = 0, totalWT = 0;

for(i = 0; i < n; i++) {
TAT[i] = CT[i] - AT[i];
WT[i] = TAT[i] - BT[i];
```

```
totalTAT += TAT[i];
totalWT += WT[i];
}

// Averages
printf("\n\nAverage Turnaround Time = %.2f\n", totalTAT / n);
printf("Average Waiting Time = %.2f\n", totalWT / n);

return 0;

}
```

OUTPUT:



Mahathi A J
241801148

Operating System

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

3

0 5 2

1 3 1

2 2 3

Sample Output 1

Enter number of processes:

Mahathi A J
241801148

Enter AT, BT and Priority for P1:

Enter AT, BT and Priority for P2:

Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67

Average Waiting Time = 3.33

Example 2

Sample Input 2

4

0 4 3

0 3 1

2 2 4

5 1 2

Sample Output 2

Enter number of processes:

Enter AT, BT and Priority for P1:

Enter AT, BT and Priority for P2:

Enter AT, BT and Priority for P3:

Enter AT, BT and Priority for P4:

Average Turnaround Time = 5.25

Average Waiting Time = 2.75

Mahathi A J
241801148

Operating System

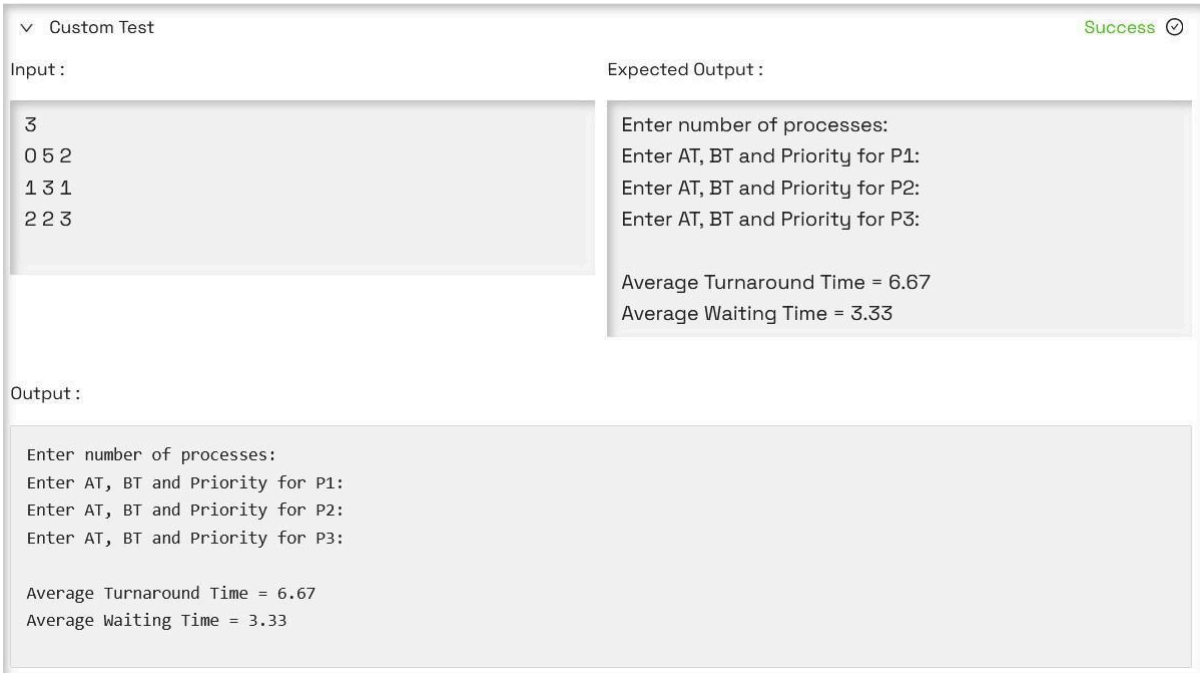
Answer:

```
#include <stdio.h>

int main() {
int n, i, time = 0, completed = 0, idx;
printf("Enter number of processes: ");
scanf("%d", &n);

int AT[n], BT[n], P[n], CT[n], TAT[n], WT[n], done[n];

for(i = 0; i < n; i++) {
printf("\nEnter AT, BT and Priority for P%d: ", i + 1);
scanf("%d %d %d", &AT[i], &BT[i], &P[i]);
done[i] = 0;
}

// Scheduling
while(completed < n) {
idx = -1;

for(i = 0; i < n; i++) {
if(AT[i] <= time && !done[i]) {
if(idx == -1 || P[i] < P[idx]) // Lower value = higher priority
idx = i;
}
}

if(idx == -1) time++; // CPU idle
else {
time += BT[idx];
CT[idx] = time;
done[idx] = 1;
completed++;
}
}

float totalTAT = 0, totalWT = 0;


for(i = 0; i < n; i++) {
TAT[i] = CT[i] - AT[i];
WT[i] = TAT[i] - BT[i];
totalTAT += TAT[i];
totalWT += WT[i];
}
printf("\n\nAverage Turnaround Time = %.2f\n", totalTAT / n);
printf("Average Waiting Time = %.2f\n", totalWT / n);
return 0;
}
```

Mahathi A J
241801148

Operating System

OUTPUT:



Mahathi A J
241801148

Operating System

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the Time Quantum.
3. The next lines contain two integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

3

2

0 5

1 3

2 1

Sample Output 1

Enter number of

processes: Enter Time

Quantum: Enter AT and

BT for P1: Enter AT and

BT for P2: Enter AT and

BT for P3:

Average Turnaround Time: 6.33

Average Waiting Time: 3.33

Mahathi A J
241801148

Operating System

Answer:

```
#include <stdio.h>

int main() {
int n, tq, i, time = 0, done = 0;
printf("Enter number of processes:
"); scanf("%d", &n);

printf("\nEnter Time Quantum: ");
scanf("%d", &tq);

int AT[n], BT[n], RT[n], CT[n];
for(i = 0; i < n; i++) {
printf("\nEnter AT and BT for P%d: ", i + 1);
scanf("%d %d", &AT[i], &BT[i]);
RT[i] = BT[i]; // Remaining Time
}

// Round Robin Scheduling
while(done < n) {
int executed = 0;

for(i = 0; i < n; i++) {
if(RT[i] > 0 && AT[i] <= time) {
executed = 1;

if(RT[i] > tq) {
time += tq;
RT[i] -= tq;
} else {
time += RT[i];
CT[i] = time;
RT[i] = 0;
done++;
}
}
}

if(!executed) time++; // CPU idle
}
float totalTAT = 0, totalWT = 0;
for(i = 0; i < n; i++) {
int TAT = CT[i] - AT[i];
int WT = TAT - BT[i];
totalTAT += TAT;
totalWT += WT;
}
```

```
printf("\n\nAverage Turnaround Time: %.2f\n", totalTAT / n);
printf("Average Waiting Time: %.2f\n", totalWT / n);

return 0;

}
```

OUTPUT: